Product Requirements Document: AI Compliance Automation Platform with vCISO

Frameworks: ISO/IEC 27001:2022 and NIST CSF 2.0 | Team: 5 people | Timeline: 3 weeks | Deliverable: working MVP and live demo

1. Overview

1.1 Product idea

A platform that helps startups and small businesses get ready for security compliance faster. It connects to the client's cloud and SaaS tools, runs automated checks, collects evidence, and maps the results to **ISO 27001:2022** and **NIST CSF 2.0**. AI drafts the security policies and answers customer security questionnaires. A **virtual CISO (vCISO)** workspace adds the human layer: a risk register, a 90-day roadmap, and leadership reports, so one security expert can guide many clients.

The pitch: *software does the evidence work, a security leader guides every decision.*

1.2 Problem

- Small companies lose enterprise deals because they cannot show security compliance.
- Existing tools give a dashboard but leave the client alone to interpret it.
- Policies, evidence, and questionnaires are slow, repetitive manual work.
- Most small companies have no in-house security leader.

1.3 Target users

User	What they need
Founder / IT manager (client admin)	See what fails, fix it, and get a compliance score
vCISO / compliance lead (our expert)	Manage several clients, risks, and roadmaps from one place
Sales / customer success at the client	Answer customer questionnaires quickly

User	What they need
Auditor (read-only)	Review evidence and control status

1.4 Important expectation

Three weeks is enough for a **working, demo-ready MVP**, not a production-ready commercial product. This document defines what "done" means for that timeline. Production hardening is listed under future improvements.

2. Goals and Success Metrics

Goal	Measure of success (end of week 3)
Prove automated compliance checking works	Platform connects to a real Microsoft 365 tenant, Google Workspace, and GitHub, and shows live pass/fail results
Prove framework mapping	Each check maps to ISO 27001 and NIST CSF controls, and the dashboard shows coverage per framework
Prove AI value	5 policies generated and reviewed; a sample questionnaire (20+ questions) autofilled with cited sources
Prove the vCISO model	One demo client has a risk register, 90-day roadmap, and generated leadership report
Deliver an auditor-ready export	One-click evidence report (PDF or CSV) for a framework

3. Frameworks Covered

Control IDs should be double-checked against the official documents by the compliance lead before the demo.

3.1 ISO/IEC 27001:2022

Focus on a **subset of Annex A (2022 version)** that can be checked technically or evidenced with documents, for example:

- Access control and identity (5.15 to 5.18, 8.2, 8.5)
- Cryptography and data protection (8.24)
- Logging and monitoring (8.15, 8.16)
- Backup (8.13)
- Vulnerability and configuration management (8.8, 8.9)
- Policies and organizational controls (5.1, 5.2, 5.37)
- Supplier relationships (5.19 to 5.22)

3.2 NIST Cybersecurity Framework 2.0

Map checks and policies to the six functions: **Govern, Identify, Protect, Detect, Respond, Recover**, at category level (for example PR.AA identity management and access control, PR.DS data security, DE.CM continuous monitoring, GV.PO policy). Category-level mapping is realistic for 3 weeks; subcategory-level mapping is future work.

3.3 How mapping works

One automated check or one approved policy links to **multiple controls across both frameworks**. Example: "MFA enforced for all users" maps to ISO 27001 access control items and to the NIST CSF Protect function's identity and access category. Store the mapping in a controls table so the data, not the code, defines it.

4. Scope (In Scope)

4.1 Core platform

1. **Authentication and roles:** login with MFA support; roles for Client Admin, vCISO, and Auditor (read-only).
2. **Multi-tenant data model:** every record belongs to an organization, enforced by PostgreSQL row-level security. The demo uses 2 to 3 organizations.
3. **Integrations (3 connectors, read-only):**
    - Microsoft 365 / Entra ID (read-only Microsoft Graph app permissions)
    - Google Workspace (users, MFA status, admin roles)

- GitHub (branch protection, 2FA for members, repo visibility)

4. **Automated checks library (25 to 30 checks)**, for example:
   - Microsoft 365 / Entra ID: MFA registered for all users, few Global Administrators, Conditional Access policies enabled, legacy authentication blocked, inactive and guest accounts reviewed, audit logging enabled
   - Google Workspace: MFA enforced, number of super admins, suspended or inactive users
   - GitHub: branch protection on main, org 2FA required, secret scanning enabled, no public repos with sensitive names
5. **Scheduler:** checks run on demand and daily.
6. **Evidence store:** each check result is saved with a timestamp, raw API response (secrets removed), and a hash for integrity.
7. **Control mapping and scoring:** pass/fail per control and a compliance percentage per framework (ISO 27001 and NIST CSF).
8. **Dashboard:** framework coverage, failing controls, plain-language fix guidance, and task assignment with status.

4.2 AI features

1. **Policy generator:** 5 policies (Information Security, Access Control, Incident Response, Backup and Recovery, Acceptable Use) drafted from a short company questionnaire, with version history and a human approval step.
2. **Questionnaire autofill:** upload a CSV or XLSX of customer security questions; AI answers each using the client's approved policies and check results, shows the source used, and flags low-confidence answers for review.
3. **Guardrails:** retrieval only from the client's own data, no model training on client data, every answer cited, nothing is marked final without human approval.

4.3 vCISO workspace

1. **Risk register:** add, score (likelihood x impact), assign owner, and track treatment, with templates.
2. **90-day roadmap:** a task plan generated from failing controls and risks, ordered by priority.
3. **Leadership report:** AI-drafted one-page quarterly summary (posture, top risks, progress), reviewed and edited by the vCISO.

4. **Multi-client view:** a single page listing each client with score, open risks, and overdue tasks.

4.4 Auditor features

- Read-only role.
- One-click export of control status and evidence for a selected framework (PDF and CSV).

5. Out of Scope (for this 3-week build)

- Formal certification or audit services; the product supports readiness only and does not guarantee a pass
- Frameworks other than ISO 27001 and NIST CSF (SOC 2, GDPR, DPDP, PCI DSS, HIPAA)
- Full coverage of every ISO or NIST control; only the defined subset
- Phishing simulation and security awareness training (the existing simulator stays separate for now)
- Automatic remediation (the platform recommends fixes but does not change client systems)
- Cloud infrastructure integrations (AWS, Azure, GCP) and other integrations; HR, ticketing, and endpoint tools
- SSO (SAML/SCIM) and enterprise identity features
- Billing, subscriptions, and payment handling
- Penetration test, formal SOC 2 for our own platform, and SLA commitments
- Mobile apps and white-labeling
- Continuous real-time monitoring (checks run on a schedule)
- Vendor risk management workflow and a public trust page
- Multi-language support and regional compliance packs
- Using the product with real production client data; the demo uses test or consenting pilot accounts only

6. Tech Stack

Layer	Choice	Reason
Frontend	Next.js (React) + TypeScript + Tailwind CSS + shadcn/ui	Fast to build polished dashboards
Backend API	Python 3.12 + FastAPI	Great for integrations and AI work, auto-generated API docs
Database	PostgreSQL 16 with row-level security	Tenant isolation at the database level
Vector search	pgvector (inside PostgreSQL)	One database for data and retrieval, less to run
Background jobs	Celery + Redis	Scheduled checks and long AI tasks
Evidence storage	S3-compatible object storage with versioning (MinIO)	Durable, auditable file storage
Authentication	Auth provider such as Clerk or Auth0 (or FastAPI Users with TOTP)	Do not build login from scratch in 3 weeks
Integrations	Microsoft Graph API (Microsoft 365), Google Admin SDK, GitHub REST API	Official SDKs, read-only scopes
AI layer	LLM API (Claude) with a retrieval-augmented generation pipeline	Policy drafting, questionnaire answers, report writing
Document handling	python-docx / WeasyPrint for PDF export, openpyxl/pandas for questionnaires	Auditor reports and uploads
Infrastructure	Docker Compose; deploy to one cloud VM or container service	Simple and demo-ready
CI/CD	GitHub Actions (lint, tests, build)	Catch mistakes early
Secrets and config	Environment secrets via cloud secrets manager or sops	Never store client credentials in code

Layer	Choice	Reason
Monitoring	Structured logs + basic uptime check (Sentry optional)	Enough for an MVP
Testing	pytest, Playwright (one end-to-end flow), a small AI evaluation set	Protect the demo path

Architecture summary

1. Client Admin connects an integration; credentials are stored encrypted and scoped read-only.
2. The scheduler (Celery) triggers each connector, which pulls data and runs checks.
3. Results and raw evidence are written to PostgreSQL and object storage.
4. The mapping layer converts check results into control status for ISO and NIST.
5. The dashboard and vCISO workspace read from the API.
6. The AI service retrieves from approved policies and evidence, generates drafts, and returns them with citations for human review.

Security requirements for our own product

- Read-only, least-privilege access for every integration
- Encryption in transit and at rest; credentials encrypted with a managed key
- Row-level security tested with automated cross-tenant access tests
- Audit log of who viewed or changed what
- No client secrets or personal data in logs or AI prompts beyond what is needed
- Dependency scanning in CI

7. Data Model (Core Tables)

Table	Purpose
organizations	Each client tenant
users, memberships	People and their roles per organization
integrations	Connected systems and encrypted credentials

Table	Purpose
frameworks, controls	ISO 27001 and NIST CSF control definitions
checks	Automated tests and their logic
check_control_map	Links each check to controls across both frameworks
check_runs, evidence	Results, timestamps, raw data references, hashes
policies, policy_versions	Drafts, approvals, version history
questionnaires, answers	Uploaded questions, AI answers, sources, review status
risks	Risk register entries
tasks	Roadmap and remediation tasks with owners and status
reports	Generated leadership reports
audit_log	Every sensitive action

8. Roles and Ownership

#	Role	Owns
1	**Product and GTM Lead**	PRD, scope control, demo script, pilot clients, auditor and vCISO contacts, pitch deck
2	**Platform / Full-stack Engineer**	Database, auth, tenancy, API, scheduler, evidence store, deployment
3	**Integrations and Security Engineer**	Microsoft 365, Google Workspace, GitHub connectors, checks library, product security, cross-tenant tests
4	**AI Engineer**	RAG pipeline, policy generator, questionnaire autofill, report generator, evaluation
5	**Compliance and vCISO Lead**	Control library and mapping, policy templates, risk register design, roadmap logic, check content review

Frontend work is shared: person 2 builds the app shell and dashboard, and persons 4 and 5 own the screens for AI features and the vCISO workspace respectively.

9. Three-Week Project Plan

Week 1: Foundation and first working slice (Days 1 to 5)

Goal: a logged-in user connects Microsoft 365, a few checks run, and the results appear on a dashboard.

Day	Product/GTM (1)	Platform (2)	Integrations/Security (3)	AI (4)	Compliance/vCISO (5)
1	Freeze scope with this PRD, set up task board, line up demo audience	Repo, Docker Compose, CI, database schema v1	Define connector interface, register read-only Microsoft 365 app	Choose model and embedding approach, set data handling rules	Draft control list (ISO subset and NIST categories)

Day	Product/GTM (1)	Platform (2)	Integrations/Security (3)	AI (4)	Compliance/vCISO (5)
2	Draft demo script and pilot test accounts	Auth, roles, org model, row-level security	Microsoft 365 connector: users, MFA, and admin roles pulled	Document ingestion and pgvector setup	Finish control library as seed data
3	Contact 2 to 3 friendly pilot companies, ask for test accounts	API for checks, evidence, controls; evidence store	First 8 Microsoft 365 checks	Prompt design for policy drafting	Check-to-control mapping table (v1)
4	Prepare questionnaire samples for testing	Scheduler (Celery) running checks	Remaining Microsoft 365 checks; start Google Workspace connector	Policy generator prototype (1 policy)	Policy templates for all 5 policies
5	Day-5 demo review, adjust scope	Dashboard v1: controls, pass/fail, scores	Google Workspace checks done	Retrieval returns cited passages	Review check content for accuracy

Week 1 exit criteria: a Microsoft 365 tenant is connected, at least 12 checks run on schedule, results map to controls, and the dashboard shows scores for both frameworks.

Week 2: Feature completion (Days 6 to 10)

Goal: all in-scope features exist in basic form.

Day	Product/GTM (1)	Platform (2)	Integrations/Security (3)	AI (4)	Compliance/vCISO (5)
6	Run internal usability session on week-1 build	Task assignment and status flow	GitHub connector and checks	Policy generator for all 5 policies	Risk register data design and scoring rules
7	Collect feedback, reprioritize	Policy editor, versions, approval step	Reach 25+ checks; failure fix-guidance text	Questionnaire upload and parsing	Roadmap generation rules from failing controls
8	Prepare pilot onboarding guide	Risk register UI (with persons 5)	Cross-tenant isolation tests	Questionnaire autofill with citations and confidence	Review AI-drafted policies for correctness

Day	Product/GTM (1)	Platform (2)	Integrations/Security (3)	AI (4)	Compliance/vCISO (5)
9	Run pilot onboarding on test accounts	vCISO multi-client page	Credential encryption, audit log	Leadership report generator	Test 90-day roadmap on pilot data
10	Mid-project review with all five	Auditor read-only role and export (CSV)	Fix issues found in pilot connection	Evaluation set: 30 questions, measure accuracy	Fix mapping gaps found in pilot

Week 2 exit criteria: all features usable end to end by a tester, one pilot or test organization is fully connected, and no known cross-tenant data leak.

Week 3: Hardening, polish, and demo (Days 11 to 15)

Goal: reliable, secure enough for a demo, and clearly presented.

Day	Product/GTM (1)	Platform (2)	Integrations/Security (3)	AI (4)	Compliance/vCISO (5)
11	Feature freeze announced	PDF export, report styling	Security review: scopes, secrets, dependencies	Tune prompts using evaluation failures	Final review of control mapping
12	Write final demo script with timing	Bug fixing, performance, empty and error states	Fix security findings	Add hallucination guards and "needs review" flags	Prepare sample client data for demo
13	Pitch deck and one-page product sheet	Deployment to demo environment	Test demo environment end to end	Final evaluation run and report	Documentation: how checks map to controls
14	Full rehearsal with all five	Backup of demo data, monitoring	Load and failure test of the connectors	Cost and latency check	Rehearse the vCISO portion
15	**Live demo and handover**	Support	Support	Support	Present vCISO workspace

Week 3 exit criteria: frozen build deployed, demo rehearsed twice, documentation and known-issues list written.

10. Definition of Done (for each feature)

- Works end to end in the demo environment
- Has an owner and at least one test
- Handles empty, error, and permission-denied states
- Does not expose another tenant's data
- Reviewed by a second team member

11. Working Rhythm

- **Daily 15-minute standup:** done yesterday, doing today, blockers
- **Day-5 and day-10 demos:** working software only
- **Scope rule:** nothing new enters after Day 11; new ideas go to the future improvements list
- **Pull requests:** at least one review before merging
- **Cut rule:** if behind schedule, cut in this order: GitHub connector, leadership report, PDF export (keep CSV), multi-client page

12. Risks and Mitigations

Risk	Likelihood	Mitigation
Scope too large for 3 weeks	High	Strict cut order above; each connector kept small
API access or permissions delays (Microsoft 365 and Google Workspace admin consent)	Medium	Create test accounts on Day 1; use own sandbox first
AI answers are wrong or fabricated	Medium	Retrieval-only answers, citations, confidence flags, mandatory human review
Wrong control mapping	Medium	Compliance lead reviews each mapping; label as "readiness guidance"
Tenant data leak	Low but severe	Row-level security plus automated cross-tenant tests

Risk	Likelihood	Mitigation
Demo-day failure	Medium	Two rehearsals, recorded backup demo, seeded demo data
Key-person dependency (one person owns compliance content)	Medium	Person 1 shadows compliance work; document decisions
Legal exposure when using real client data	Medium	Use test or consenting accounts only; no certification claims

13. Future Improvements

Near term (1 to 3 months)

- Add SOC 2, GDPR, and India's DPDP Act mappings
- More integrations: cloud infrastructure (AWS, Azure, GCP), Slack, Jira, HR systems, endpoint tools
- Remediation pull requests and one-click fixes with approval
- SSO (SAML) and fine-grained permissions
- Evidence immutability with object lock and signed audit trails
- Penetration test and our own security certification

Mid term (3 to 9 months)

- Reintroduce employee security training and the phishing simulator, with training records feeding evidence
- Vendor risk management and a public trust page for the client's customers
- Auditor marketplace and partnered fixed-price audits
- Real-time monitoring and alerting
- Billing, subscription plans, and customer self-service onboarding
- Better AI: fine-tuned evaluation, per-industry policy packs, multilingual support

Longer term

- Full subcategory-level NIST mapping and complete ISO Annex A coverage
- Continuous control monitoring across multi-cloud
- White-label version for MSSPs and consultants
- Benchmarking against similar companies

- AI security gateway module for clients shipping LLM features

14. Open Questions for the Team

1. Which cloud accounts and test environments do we have access to on Day 1?
2. Who is the named vCISO or compliance reviewer for the demo, and can they commit time?
3. Do we have 1 to 2 friendly companies for pilot testing, and have they agreed in writing?
4. Which LLM provider and budget cap are we using for development and the demo?
5. Is the demo audience investors, potential clients, an instructor, or a mix? This shapes the script.

15. Appendix: Demo Script (10 minutes)

1. **Problem (1 min):** a startup losing a deal because of a missing security report.
2. **Connect (1 min):** connect Microsoft 365, Google Workspace, and GitHub in the dashboard.
3. **Results (2 min):** live checks, failing controls, and scores for ISO 27001 and NIST CSF.
4. **Policies (2 min):** generate and approve an Access Control policy.
5. **Questionnaire (2 min):** upload a customer questionnaire, show autofilled and cited answers.
6. **vCISO (1 min):** risk register, 90-day roadmap, and multi-client view.
7. **Auditor export (1 min):** download the evidence report and close with next steps.

16. Claude Code Build Prompt

Paste the master prompt below into Claude Code at the root of an empty repo, then run the phase prompts one at a time. Save the master prompt's rules in a CLAUDE.md file so every session follows them. Review each phase's plan and diff before approving; Claude Code speeds up the work but does not replace code review, especially for security and tenant isolation.

16.1 Master prompt

```
You are a senior full-stack and security engineer. Help me build an MVP
called "AI Compliance Automation Platform with vCISO" in 3 weeks. Work in
phases. Before writing code in each phase, show a short plan and wait for my
approval.

PRODUCT
A multi-tenant web platform for startups and SMBs preparing for ISO/IEC
27001:2022 and NIST CSF 2.0. It connects to a client's Microsoft 365 / Entra
ID, Google Workspace and GitHub using read-only access, runs automated
checks, stores evidence, maps results to controls in both frameworks, and
shows compliance scores. AI drafts policies and answers security
questionnaires with cited sources. A vCISO workspace provides a risk
register, a 90-day roadmap, a multi-client view and an AI-drafted leadership
report. Auditors get a read-only role and an evidence export.

TECH STACK (use exactly this unless I approve a change)
- Frontend: Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui
- Backend: Python 3.12, FastAPI, SQLAlchemy 2.x, Alembic, Pydantic v2
- Database: PostgreSQL 16 with row-level security, pgvector extension
- Jobs: Celery with Redis (scheduled and on-demand check runs)
- Object storage: S3-compatible (MinIO in local dev)
- Auth: an external auth provider (Clerk or Auth0) with MFA support; roles:
client_admin, vciso, auditor
- Integrations: Microsoft Graph API, Google Admin SDK, GitHub REST API (read-
only scopes only)
- AI: Anthropic Claude API with a retrieval step over the tenant's own
policies and evidence; embeddings stored in pgvector
- Exports: WeasyPrint for PDF, CSV for data
- Tooling: Docker Compose, GitHub Actions (lint, type-check, tests), pytest,
Playwright for one end-to-end flow

REPO STRUCTURE
/apps/web (Next.js), /apps/api (FastAPI), /apps/worker (Celery tasks),
/packages/connectors, /db/migrations, /seed (frameworks, controls, checks,
mappings), /docs, docker-compose.yml, CLAUDE.md

NON-NEGOTIABLE RULES
1. Every tenant-owned table has an organization_id and PostgreSQL row-level
security. The API sets the tenant context per request. Write automated tests
proving tenant A cannot read or write tenant B data.
2. Connectors are read-only and least privilege. Never request write scopes.
```

```
Store credentials encrypted; never log secrets, tokens or raw personal data.
3. Never store passwords or secrets in code, seed data or test fixtures. Use
environment variables and a .env.example.
4. The AI layer only retrieves from the requesting tenant's data. Every
generated policy, questionnaire answer and report must include citations to
its sources and a confidence flag, and must stay in draft until a human
approves it. Never present AI output as final or as a guarantee of
certification.
5. Controls, checks and mappings are data (seed files and tables), not hard-
coded logic. Adding a check or a mapping must not require changing core code.
6. Every sensitive action writes to audit_log (who, what, when, tenant).
7. Validate all input, use parameterized queries, and handle empty, error and
permission-denied states in the UI.
8. Write tests alongside features. Do not mark a task done until tests pass
and the app runs with docker compose up.
9. Keep scope tight. If something is not in the scope list below, do not
build it; add it to /docs/future.md instead.
10. Use only fictional sample data in tests and demos.

IN SCOPE
- Auth, organizations, roles (client_admin, vciso, auditor)
- Connectors: Microsoft 365 / Entra ID, Google Workspace, GitHub
- 25 to 30 automated checks (for example MFA registered for all users, number
of Global Administrators, Conditional Access enabled, legacy authentication
blocked, inactive and guest accounts, org 2FA required on GitHub, branch
protection on default branches, secret scanning enabled)
- Scheduler (daily and on-demand) and an evidence store (timestamp, sanitized
raw response, SHA-256 hash)
- Control library: a subset of ISO 27001:2022 Annex A and NIST CSF 2.0 at
category level, with a check-to-control mapping table
- Dashboard: score per framework, failing controls with plain-language fix
guidance, task assignment and status
- AI: generate 5 policies (Information Security, Access Control, Incident
Response, Backup and Recovery, Acceptable Use) with versions and approval;
questionnaire upload (CSV/XLSX) with autofilled, cited answers and low-
confidence flags
- vCISO workspace: risk register (likelihood x impact), 90-day roadmap
generated from failing controls and risks, multi-client overview, AI-drafted
leadership report
- Auditor role: read-only access and evidence export (PDF and CSV)

OUT OF SCOPE (do not build)
SOC 2/GDPR/DPDP mappings, AWS/Azure/GCP connectors, phishing simulation and
```

```
training, auto-remediation, SSO/SAML, billing, mobile apps, vendor risk
workflow, public trust page, real-time monitoring.

BUILD ORDER
Phase 1: repo, Docker Compose, CI, database schema, migrations, row-level
security, auth and roles, tenant isolation tests.
Phase 2: connector interface plus the Microsoft 365 connector, check engine,
evidence store, scheduler, seed controls and mappings, scoring.
Phase 3: dashboard, task flow, Google Workspace and GitHub connectors,
remaining checks, fix guidance.
Phase 4: retrieval pipeline, policy generator, questionnaire autofill,
evaluation set of 30 questions.
Phase 5: vCISO workspace (risk register, roadmap, multi-client view,
leadership report), auditor role and exports.
Phase 6: security review, bug fixing, seeded demo data, deployment, README
and demo script.

HOW TO WORK
- Start by reading this prompt, then propose the Phase 1 plan: files to
create, data model, and test plan. Wait for my approval.
- After each phase, summarize what was built, how to run it, what tests
cover, and any open risks or assumptions.
- If a requirement is unclear, ask one concise question rather than guessing.
```

16.2 Phase prompts

Run these in order, one per working session. Each assumes the master prompt rules are saved in CLAUDE.md.

Phase	Prompt to give Claude Code
1. Foundation	Implement Phase 1. Create the repo structure, Docker Compose (api, web, worker, postgres with pgvector, redis, minio), CI workflow, Alembic migrations for the core tables (organizations, users, memberships, integrations, frameworks, controls, checks, check_control_map, check_runs, evidence, policies, policy_versions, questionnaires, answers, risks, tasks, reports, audit_log), row-level security policies, and role-based access. Add pytest tests that prove cross-tenant reads and writes are blocked.
2. Checks engine	Implement Phase 2. Build the connector interface (authenticate, fetch, normalize) and the Microsoft 365 connector using read-only Graph permissions. Build the check engine that loads checks from seed data, runs them, stores sanitized evidence with a SHA-256 hash, and computes control status and a score per framework. Add Celery scheduling (daily and on-demand). Seed the ISO 27001 and NIST CSF control subsets and check mappings from files in /seed. Use recorded fixtures so tests run without real credentials.
3. Dashboard and connectors	Implement Phase 3. Build the dashboard (framework scores, failing controls, fix guidance, task assignment and status). Add the Google Workspace and GitHub connectors with their checks until the library reaches 25 to 30 checks. Add empty, loading and error states and one Playwright test covering connect, run checks and view results.

Phase	Prompt to give Claude Code
4. AI features	Implement Phase 4. Build the retrieval pipeline over a tenant's approved policies and latest evidence using pgvector. Build the policy generator for the 5 policies from a short company questionnaire, with versions and an approval step. Build questionnaire upload and autofill with citations and confidence flags. Create an evaluation script with 30 sample questions and report accuracy and the share of answers flagged for review. Enforce that retrieval never crosses tenants.
5. vCISO and auditor	Implement Phase 5. Build the risk register (likelihood x impact, owner, treatment), the 90-day roadmap generator from failing controls and risks, the multi-client overview page, and the AI-drafted leadership report with citations and an approval step. Add the auditor read-only role and PDF and CSV evidence export per framework.
6. Hardening	Implement Phase 6. Run a security review of the code: authorization on every endpoint, secrets handling, dependency scanning, input validation, logging of sensitive data, and tenant isolation tests. Fix what you find. Create seeded demo data with fictional companies, a deployment guide, a README, and a 10-minute demo script matching the PRD. List known issues in /docs/known-issues.md.

16.3 Tips for working with Claude Code on this project

- **Plan first, then code.** Ask for the plan, edit it, and only then approve implementation.
- **Keep sessions small.** One phase or one feature per session; clear the context between phases and rely on CLAUDE.md and the docs folder for memory.
- **Use fixtures, not real accounts,** in automated tests; test with a real Microsoft 365 tenant only manually and with test users.
- **Review security-sensitive code yourself:** row-level security policies, credential handling, and anything that touches AI prompts.

- **Commit after every working step** so you can roll back a bad change quickly.
- **Track the PRD cut order** (GitHub connector, leadership report, PDF export, multi-client page) and tell Claude Code explicitly when to skip a feature.